

Український державний університет імені Михайла Драгоманова

Факультет математики, інформатики та фізики

Кафедра комп'ютерної та програмної інженерії

Технічний звіт

з курсу

«Розробка мобільних застосунків»

Роботу виконав

студент 4 курсу

групи 41ПЗ

Шевцов Богдан Олександрович

Перевірив:

Шевчук Б.В.

Київ-2026

Зміст

ВСТУП	3
ОГЛЯД ПРЕДМЕТНОЇ ОБЛАСТІ	3
АРХІТЕКТУРА РІШЕННЯ	3
ДЕМОНСТАРЦЯ РОБОТИ.....	6
РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА ПРОФІЛЮВАННЯ.....	7
ВИСНОВКИ.....	9

ВСТУП

Метою даної роботи є практичне дослідження методів зберігання даних на мобільних пристроях під управлінням ОС Android за допомогою бібліотеки Room (версія 2.6.1). У рамках роботи було розроблено застосунок «Замітки» (Notes App), який дозволяє користувачеві створювати, переглядати, шукати, редагувати та видаляти текстові записи.

Актуальність роботи зумовлена тим, що забезпечення роботи застосунків в офлайн-режимі (offline-first підхід) є стандартом сучасної мобільної розробки. Використання бібліотеки Room дозволяє абстрагуватися від низькорівневих SQL-запитів, забезпечуючи перевірку коду на етапі компіляції та зручну інтеграцію з архітектурними компонентами Android (Lifecycle, LiveData).

ОГЛЯД ПРЕДМЕТНОЇ ОБЛАСТІ

Для зберігання структурованих даних в Android традиційно використовується реляційна база даних SQLite. Однак пряма робота з SQLiteOpenHelper вимагає написання великої кількості шаблонного (boilerplate) коду та не має вбудованої перевірки SQL-запитів, що часто призводить до помилок під час виконання програми (Runtime exceptions).

Для вирішення цих проблем Google представила Room — ORM (Object-Relational Mapping) бібліотеку, яка є частиною Android Jetpack. Room створює абстрактний шар над SQLite, дозволяючи розробникам працювати з базою даних через об'єкти мови Java/Kotlin.

Серед конкурентів Room можна виділити Realm (NoSQL рішення з високою швидкістю), SQLDelight (генерує код із SQL-запитів, популярний у кросплатформній розробці) та ObjectBox. Однак Room залишається де-факто стандартом для нативної Android-розробки завдяки офіційній підтримці, легкій кривій навчання та безшовній інтеграції з архітектурним патерном MVVM.

АРХІТЕКТУРА РІШЕННЯ

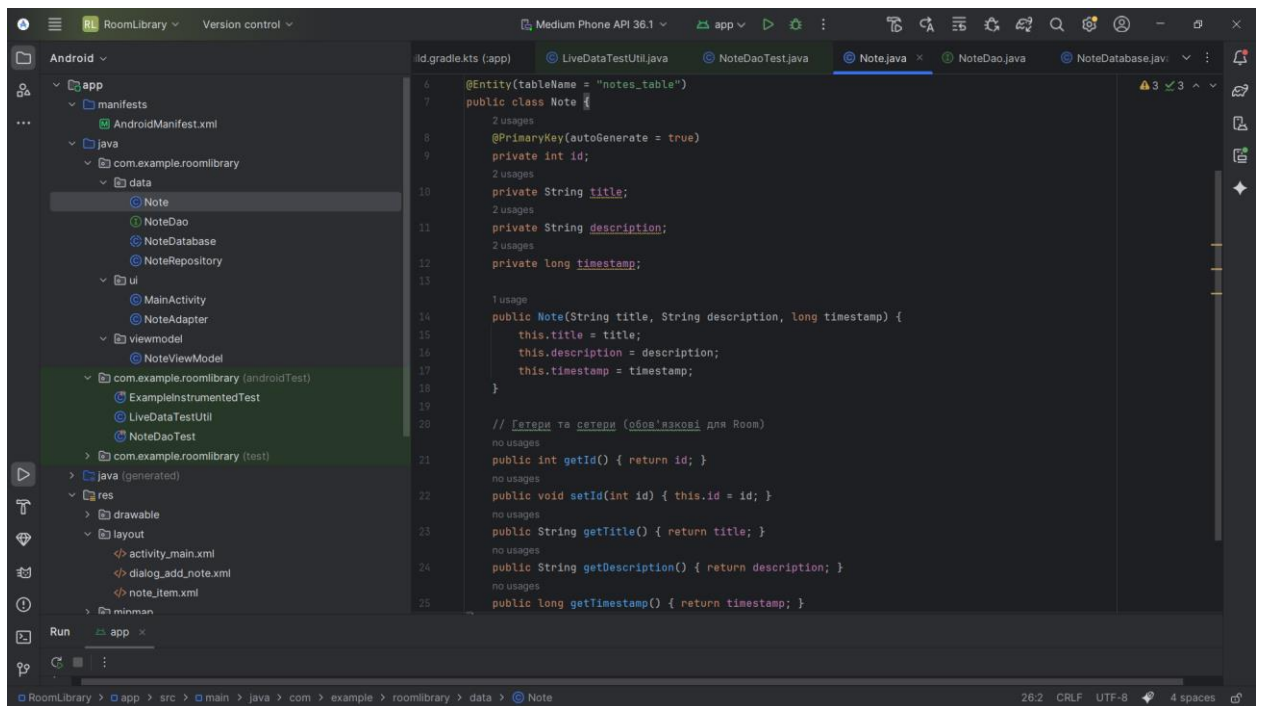
Застосунок побудовано за принципами чистої архітектури з використанням патерну MVVM (Model-View-ViewModel). Це забезпечує розділення бізнес-логіки та інтерфейсу користувача, роблячи код гнучким до змін та зручним для тестування.

Архітектура складається з наступних рівнів:

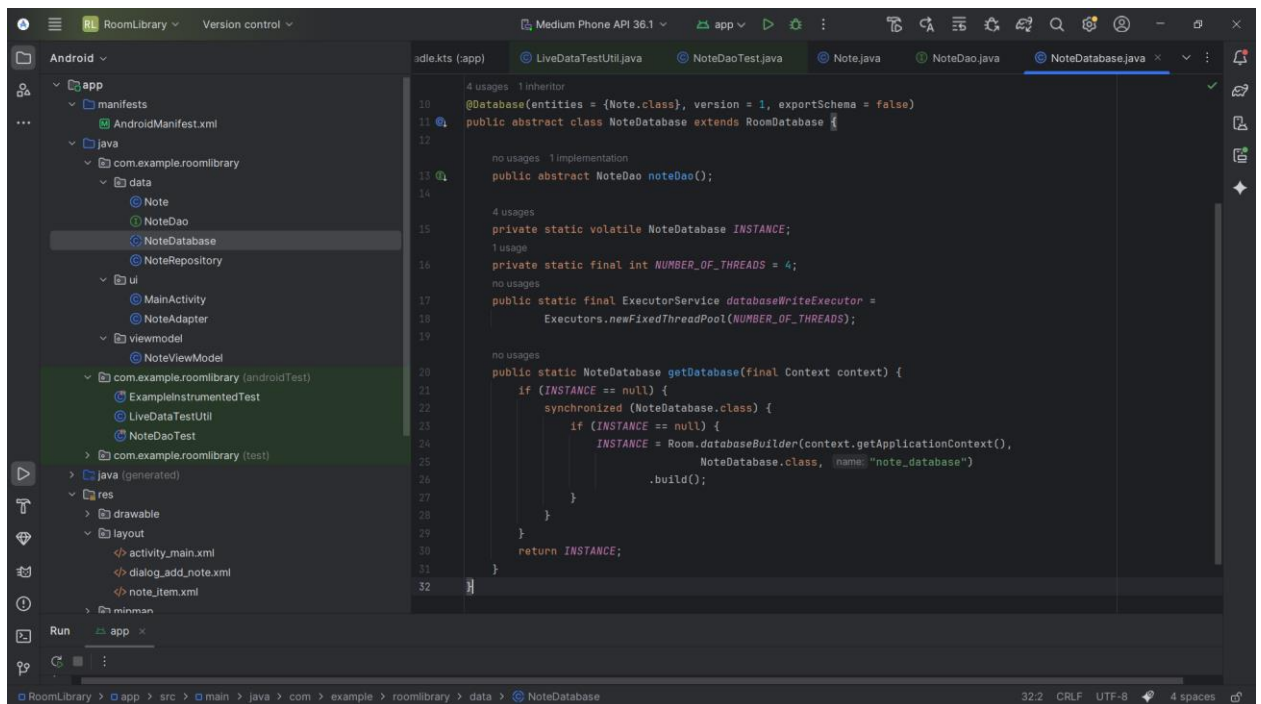
1. Model (Data Layer):

Entity (Note.java): Клас даних, анотований @Entity, що представляє таблицю notes_table у базі даних. Містить поля: id, title, description, timestamp.

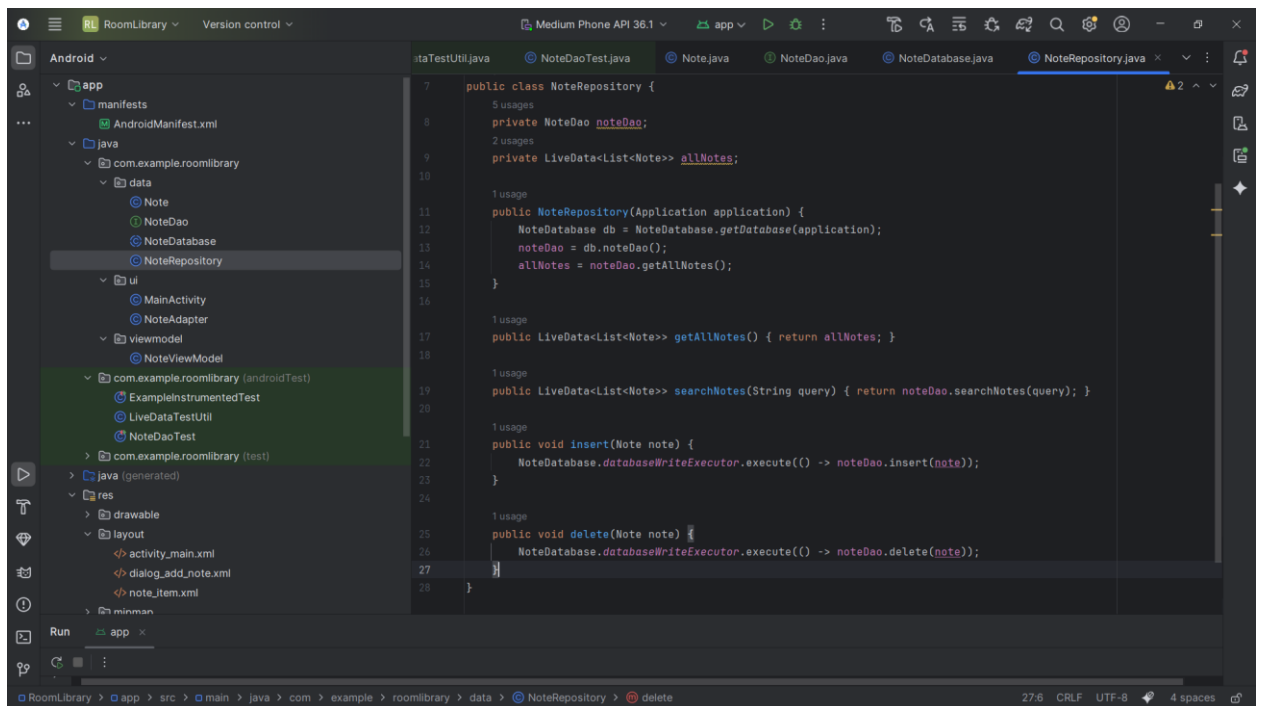
DAO (NoteDao.java): Інтерфейс (Data Access Object), який містить SQL-запити для виконання базових (CRUD) та просунутих операцій (пошук LIKE, сортування ORDER BY). Повертає об'єкт LiveData для реактивного оновлення UI.



Database (NoteDatabase.java): Головний клас бази даних, реалізований за патерном Singleton. Включає ExecutorService з пулом на 4 потоки для виконання операцій запису/видалення у фоновому режимі, що запобігає блокуванню головного потоку (UI Thread) і помилкам ANR (Application Not Responding).

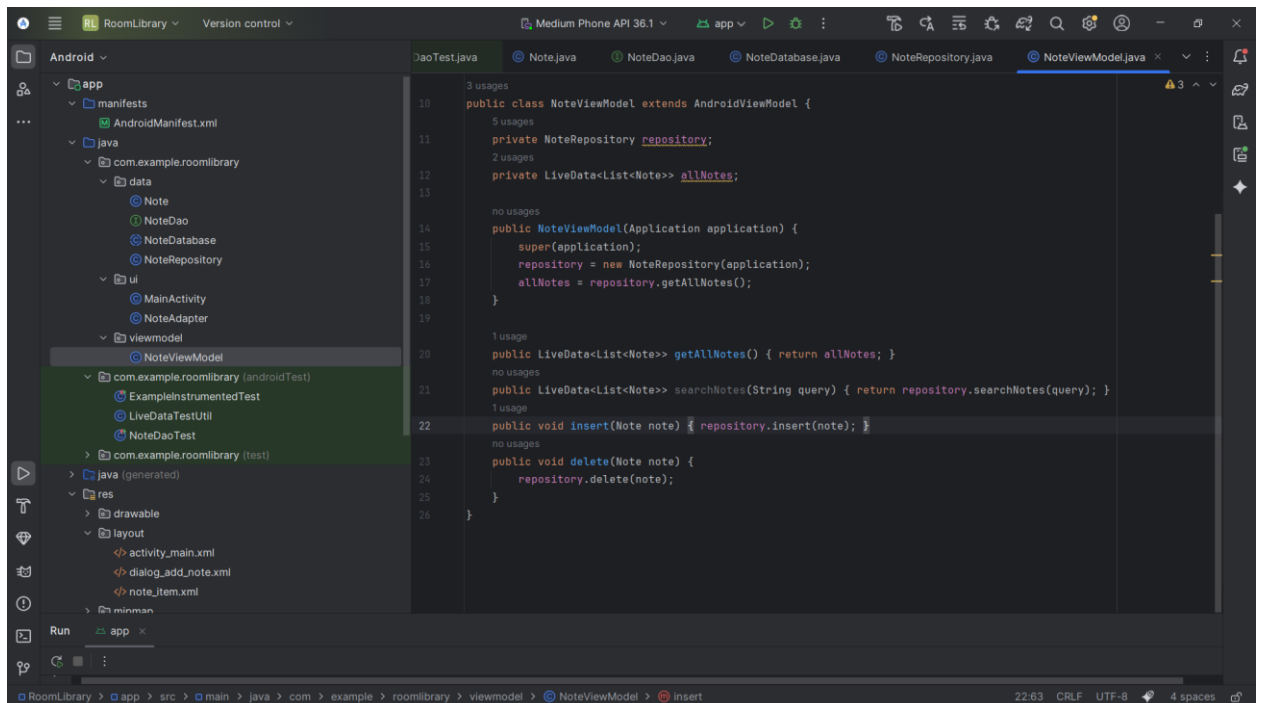


Repository (NoteRepository.java): Клас, що абстрагує джерела даних. Він є єдиною точкою доступу до даних для ViewModel.



2. ViewModel (NoteViewModel.java):

Зберігає та обробляє дані для UI. Вона переживає зміни конфігурації (наприклад, поворот екрану), тому дані не втрачаються.

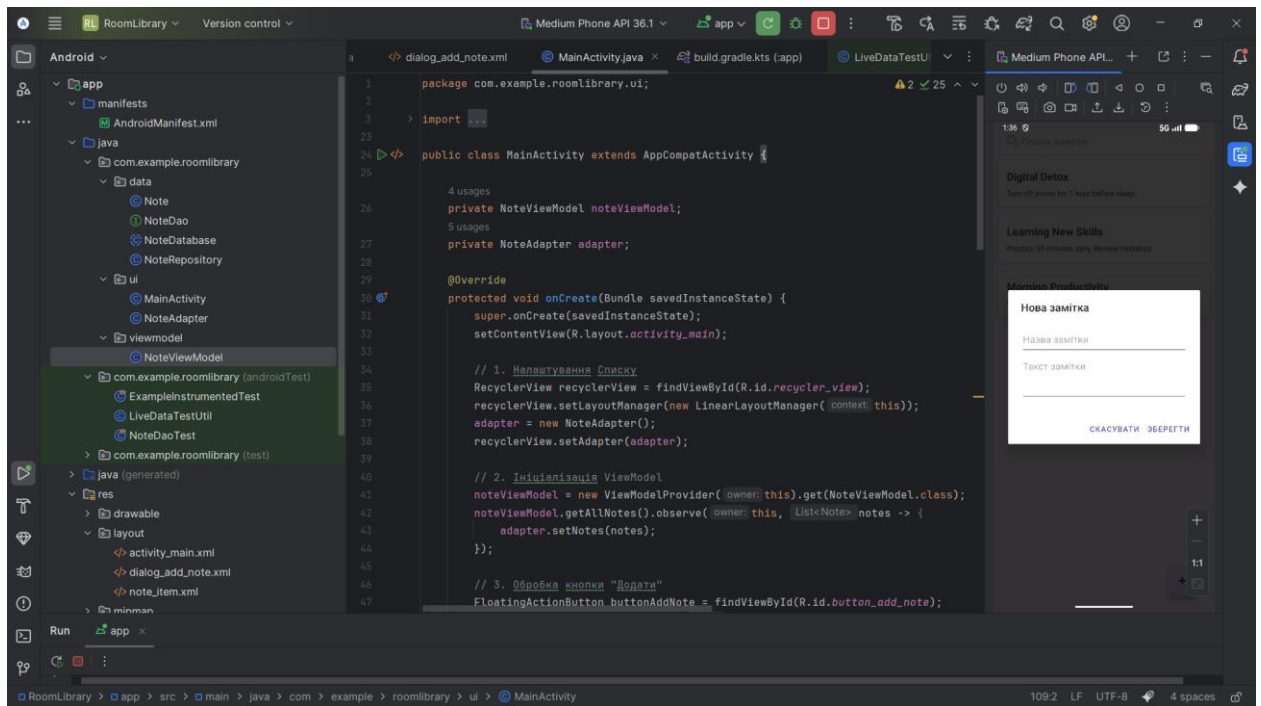


3. View (UI Layer):

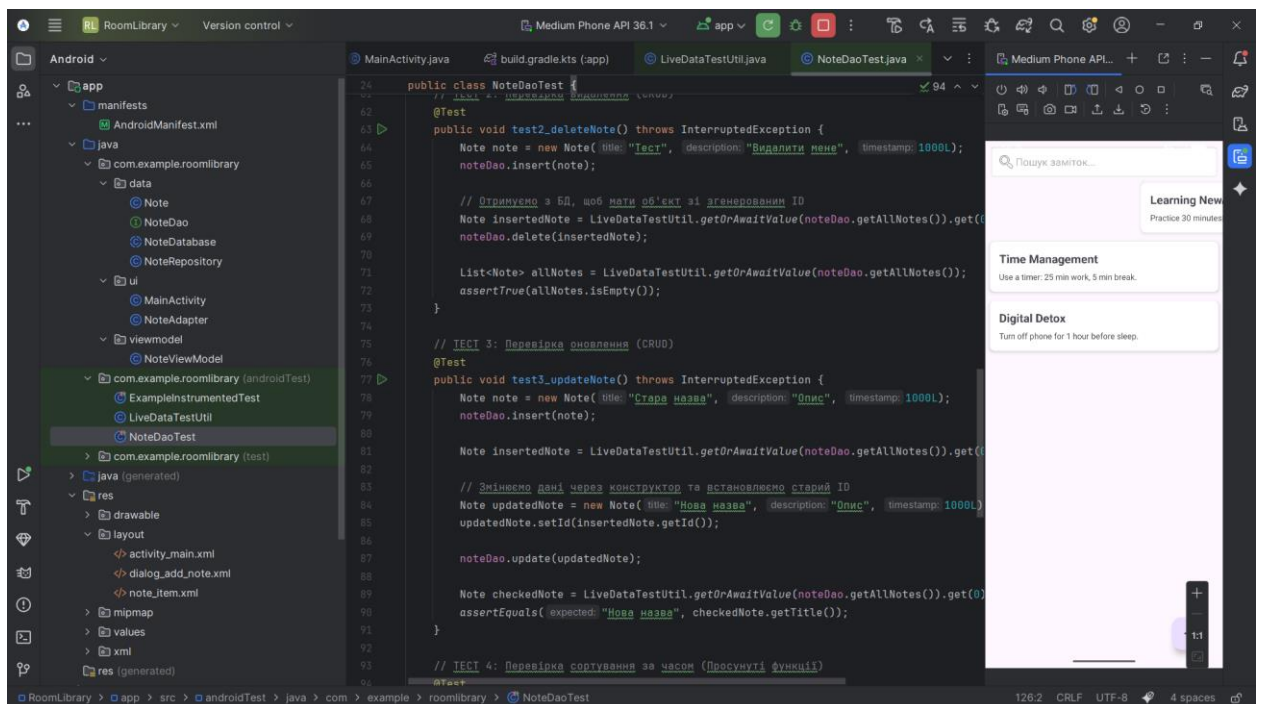
MainActivity та NoteAdapter підписані на зміни в LiveData. При зміні даних у базі, інтерфейс (RecyclerView) оновлюється автоматично. Реалізовано обробку жестів (ItemTouchHelper) для видалення заміток свайпом та TextWatcher для динамічного пошуку.

ДЕМОНСТРАЦІЯ РОБОТИ

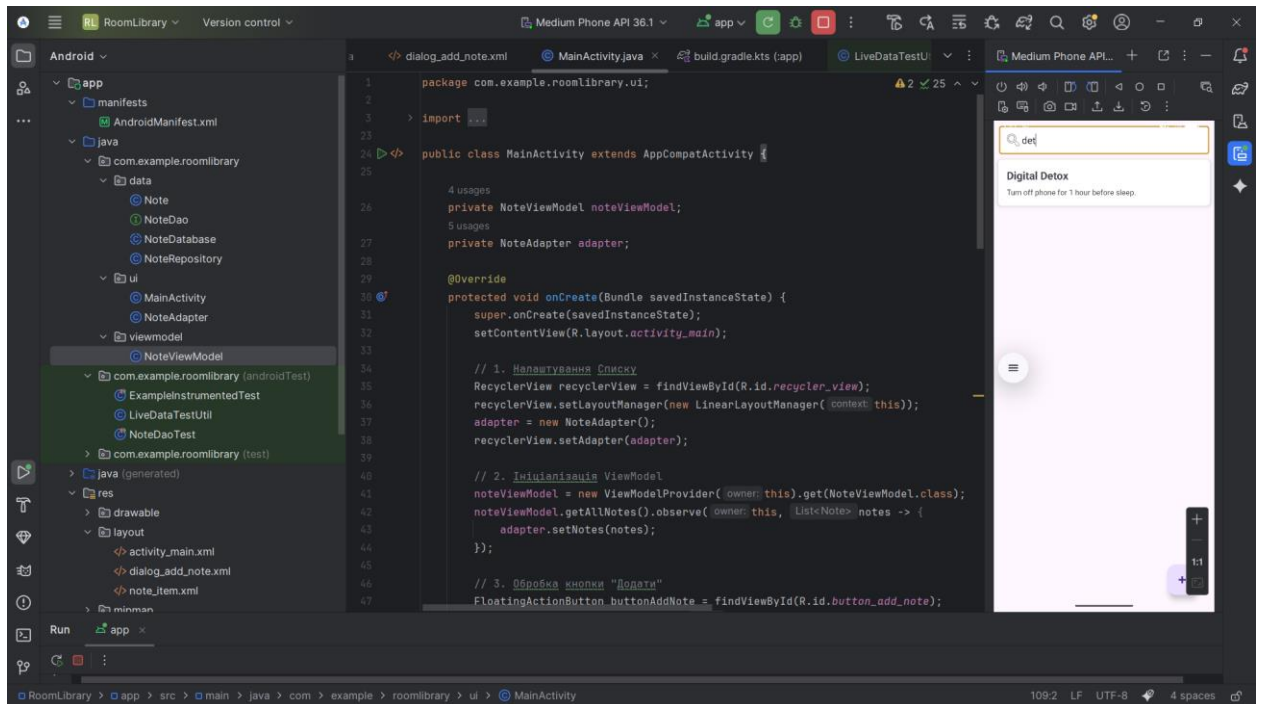
Додавання нової замітки:



Видалення перетягуванням в сторону:



Пошук:



РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА ПРОФІЛЮВАННЯ

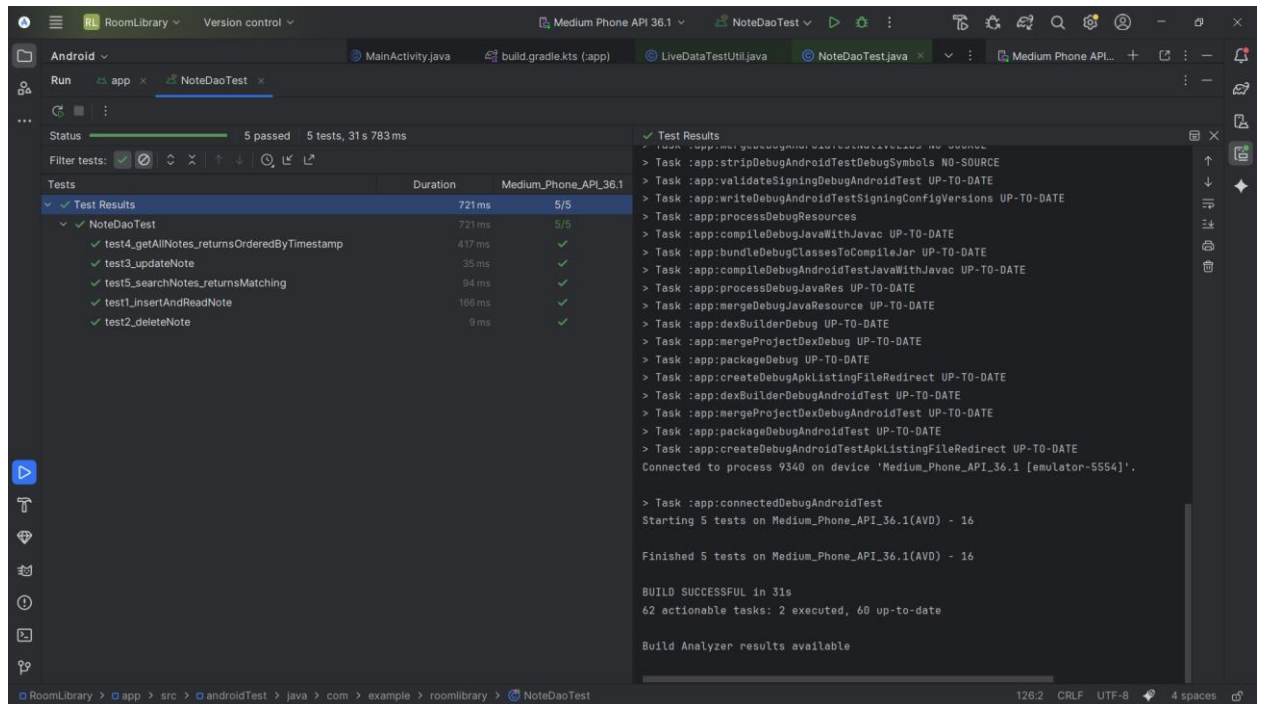
Для перевірки працездатності бізнес-логіки було написано 5 Unit/Інтеграційних тестів за допомогою фреймворків JUnit 4 та AndroidX Room Testing.

Проведені тести:

1. Вставка та зчитування даних (перевірка базового створення).
2. Видалення запису за ідентифікатором.
3. Оновлення існуючого запису.
4. Перевірка сортування (записи повертаються у порядку спадання timestamp).
5. Пошук за ключовим словом (перевірка фільтрації списку).

Усі тести виконувалися з використанням In-Memory бази даних, що ізолює тестове середовище від реальних даних пристрою. Усі 5 тестів пройшли успішно.

Пройдені тести:



Профілювання застосунку (Метрики):

Для збору метрик використовувався інструмент Android Studio Profiler на цільовому пристрої.

Завдяки оптимізації та мінімальній кількості сторонніх бібліотек, базовий розмір APK складає приблизно 5 МБ. Сама бібліотека Room додає до загального розміру менше 100 КБ.

У стані спокою застосунок займає близько 45 МБ оперативної пам'яті. Під час активного скролінгу та пошуку споживання зростає незначно, оскільки RecyclerView ефективно перевикористовує пам'ять (ViewHolders).

Ініціалізація бази даних відбувається асинхронно. Пікове навантаження на процесор спостерігається лише під час збереження нових записів та виконання повнотекстового пошуку.

ВИСНОВКИ

У ході виконання завдання було успішно інтегровано бібліотеку Room версії 2.6.1 у нативний Android-застосунок на мові Java.

Розроблений застосунок повністю відповідає поставленим вимогам:

- Реалізовано базові CRUD-операції.
- Додано просунуті функції: сортування за часом та повнотекстовий пошук у реальному часі.
- Опрацьовано edge cases: застосунок є повністю автономним (не потребує мережі), а всі важкі операції з базою даних делеговані у фонові потоки, що гарантує плавність інтерфейсу.
- Архітектура MVVM забезпечила надійність роботи при змінах конфігурації екрану.
- Написані інтеграційні тести підтвердили коректність SQL-запитів та логіки DAO.

Порівняльний аналіз показав, що Room є найбільш оптимальним вибором для цього класу задач завдяки надійності, відсутності повторення коду та нативній підтримці від розробників ОС Android.